Input signals: waves and voices

The starting point of all audio signal processing is the measured microphone signal. This can be either a wave file or the ouput of a microphone driver. In the air, audio signals are changes in pressure and velocity. These changes are received by microphones or the human ear. Before the digital signal processing can be startet, the microphone signal $x(t)$ is converted from analogue world into the digital world by the analogue-digital-converter (ADC). The ADC can be described by three parameters:

Overload point A: the maximum analogue magnitude which can be converted without distortion. If magnitudes above the overload point are measured, clipping occurs. All magnitudes of $x(t)$ lie in the range $-A..A$ after quantisation.

Sampling rate r: the sampling rate states how many samples are measured per second. In Audio processing, common used sampling rates are $r = 44100$ Hz (Audio CD), $r = 48000$ Hz (professional audio equipment), $r = 16000$ Hz (wideband audio) or $r = 8000$ Hz (narrowband audio). The inverse of the sampling rate is the sampling duration $T = \frac{1}{r}$, which corresponds to the temporal distance between two neighbouring samples.

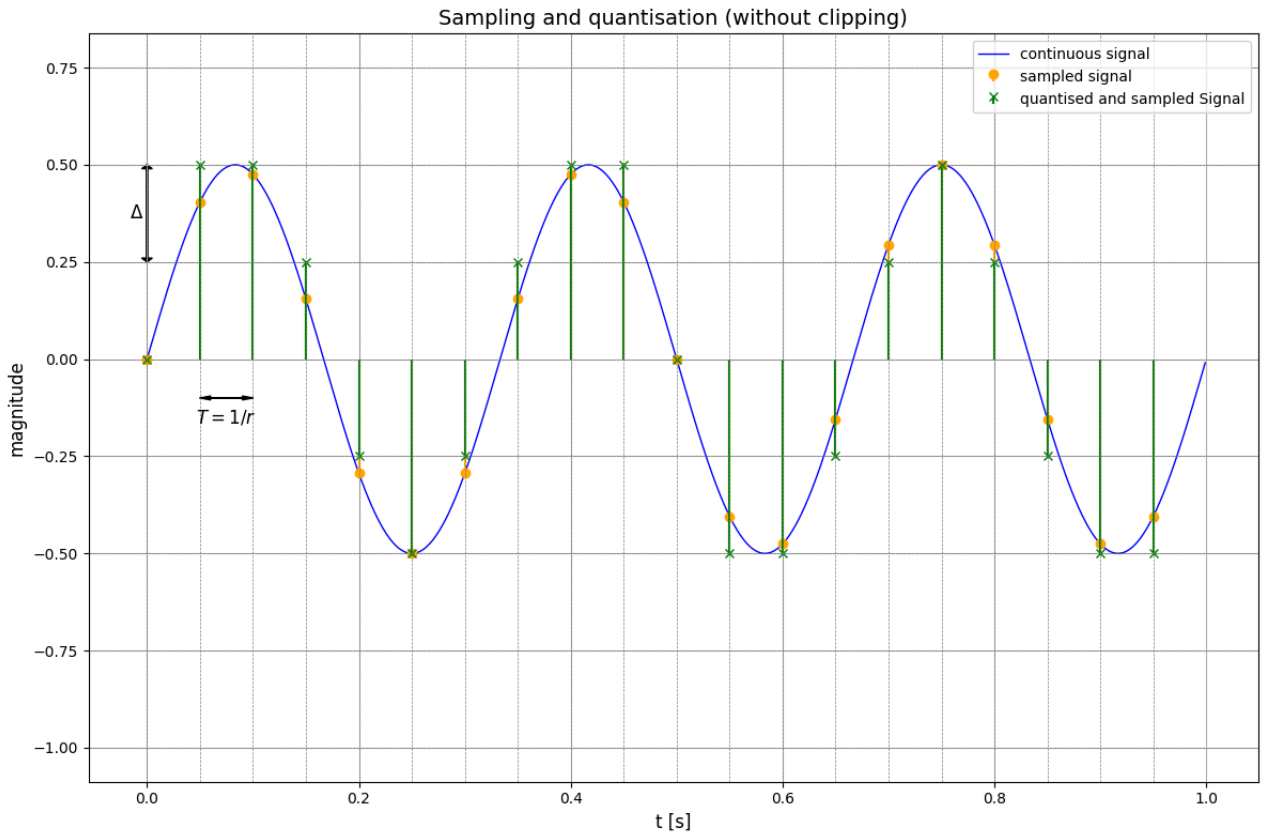
Number of bits w: the quantisation uses a defined number of bits per audio sample. Therefore, 2^w different quantisation levels of the magnitudes are possible. This leads to a quantisation step $\Delta = \frac{2A}{2^w}$.

The product of sampling rate r and number of bits w gives the amound of data per audio channel and second.

[1]

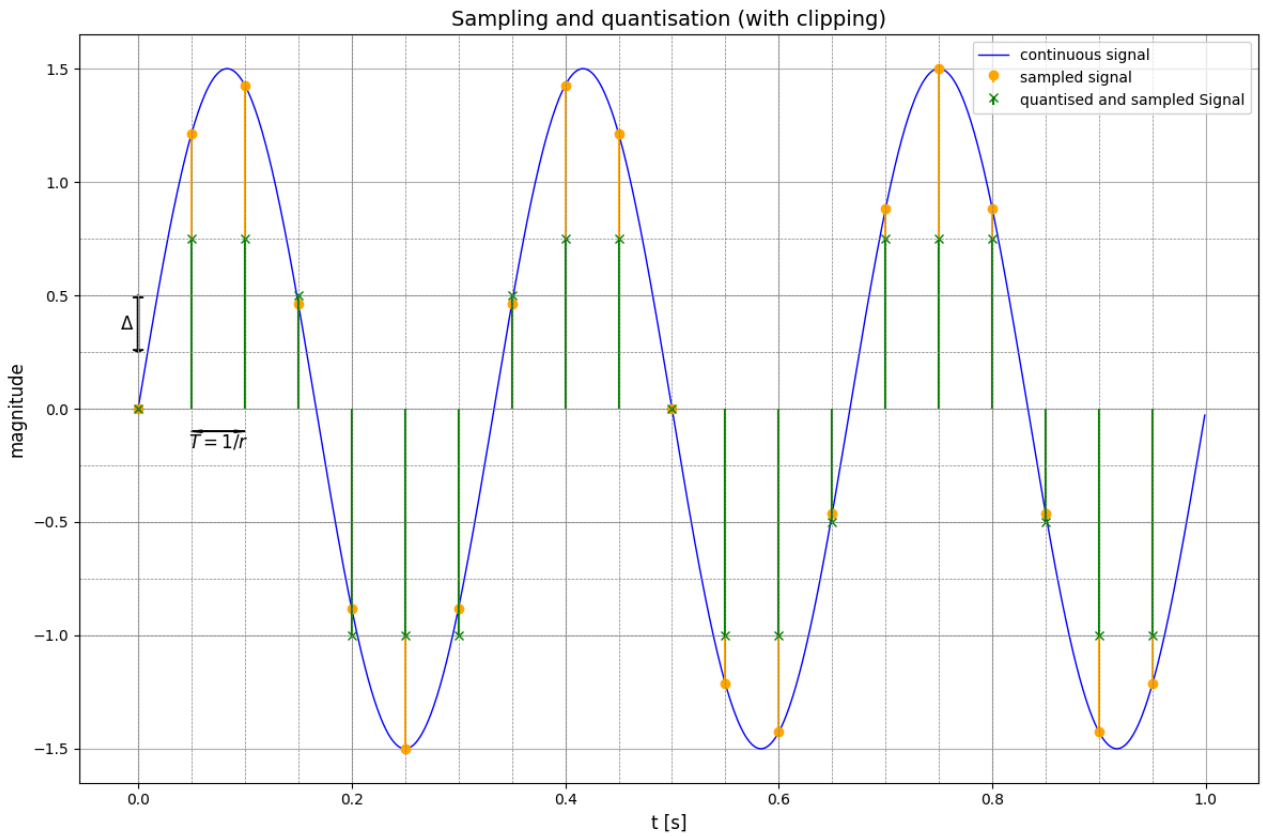
```
import numpy as np
import matplotlib.pyplot as plt

def CreatePlot(A, fs_cont, t_cont, signal_cont, title="Sampling and quantisation
(without clipping)"):
    r = 20  # low sampling rate for visualizing sampling and quantisation
    SamplingFactor = int(fs_cont / r)
    t_sampled = t_cont[::SamplingFactor]
    signal_sampled = signal_cont[::SamplingFactor]

    w = 3
    quantization_levels = 2**w  # Anzahl der Quantisierungsstufen
    delta = (2*A) / quantization_levels
    #signal_quantized = np.round(signal_sampled * (quantization_levels / 2)) /
(quantization_levels / 2)
    signal_quantized = np.round(signal_sampled / delta)
    signal_quantized = np.minimum(signal_quantized, quantization_levels // 2 - 1)  #
Clip to max level
    signal_quantized = np.maximum(signal_quantized, -quantization_levels // 2)  #
Clip to min level
    signal_quantized = signal_quantized * delta

    plt.figure(figsize=(12, 8))
    plt.plot(t_cont, signal_cont, label="continuous signal", color="blue",
linewidth=1)
    plt.stem(t_sampled, signal_sampled, linefmt="orange", markerfmt="o", basefmt="
", label="sampled signal")
```

```
    plt.stem(t_sampled, signal_quantized, linefmt="green", markerfmt="x", basefmt="
", label="quantised and sampled Signal")

    # mark all quantisation intervals
    for t in t_sampled:
        plt.axvline(x=t, color='gray', linestyle='--', linewidth=0.5)
    value = -A
    while value < A:
        plt.axhline(value, color='gray', linestyle='--', linewidth=0.5)
        value += delta

    # plot quantisation level Δ
    plt.arrow(0, delta, 0, delta, head_width=0.01, head_length=0.01, linewidth=1,
color='k', length_includes_head=True)
    plt.arrow(0, 2*delta, 0, -delta, head_width=0.01, head_length=0.01, linewidth=1,
color='k', length_includes_head=True)
    plt.text(-0.01, 1.5*delta, r"$\Delta$", ha="center", va="center", fontsize=12,
color="black")
    # plotting of sampling rate and sampling duration
    plt.arrow(1/r, -0.1, 1/r, 0, head_width=0.01, head_length=0.01, linewidth=1,
color='k', length_includes_head=True)
    plt.arrow(2/r, -0.1, -1/r, 0, head_width=0.01, head_length=0.01, linewidth=1,
color='k', length_includes_head=True)
    plt.text(1.5/r, -0.15, r"$T=1/r$", ha="center", va="center", fontsize=12,
color="black")

    plt.title(title, fontsize=14)
    plt.xlabel("t [s]", fontsize=12)
    plt.ylabel("magnitude", fontsize=12)
    plt.legend(fontsize=10)
    plt.grid(True)
    plt.tight_layout()
    plt.show()

A = 1.0  # overload point
f = 3  # frequency of the sinusoidal signal
fs_cont = 1000  # high sampling rate for continuous signal
t_cont = np.linspace(0, 1, fs_cont, endpoint=False)
signal_cont = 0.5*A*np.sin(2 * np.pi * f * t_cont)
CreatePlot(A, fs_cont, t_cont, signal_cont)
```



[2]

```
signal_cont = 1.5*A*np.sin(2 * np.pi * f * t_cont)
CreatePlot(A, fs_cont, t_cont, signal_cont, title="Sampling and quantisation (with
clipping)")
```



Sampling Theorem

The sampling theorem states, that the highest frequency f_c of the time-continuous signal $x(t)$ must be lower or equal than half of the samplerate:

$$f_c \leq \frac{1}{2T} = \frac{r}{2}$$

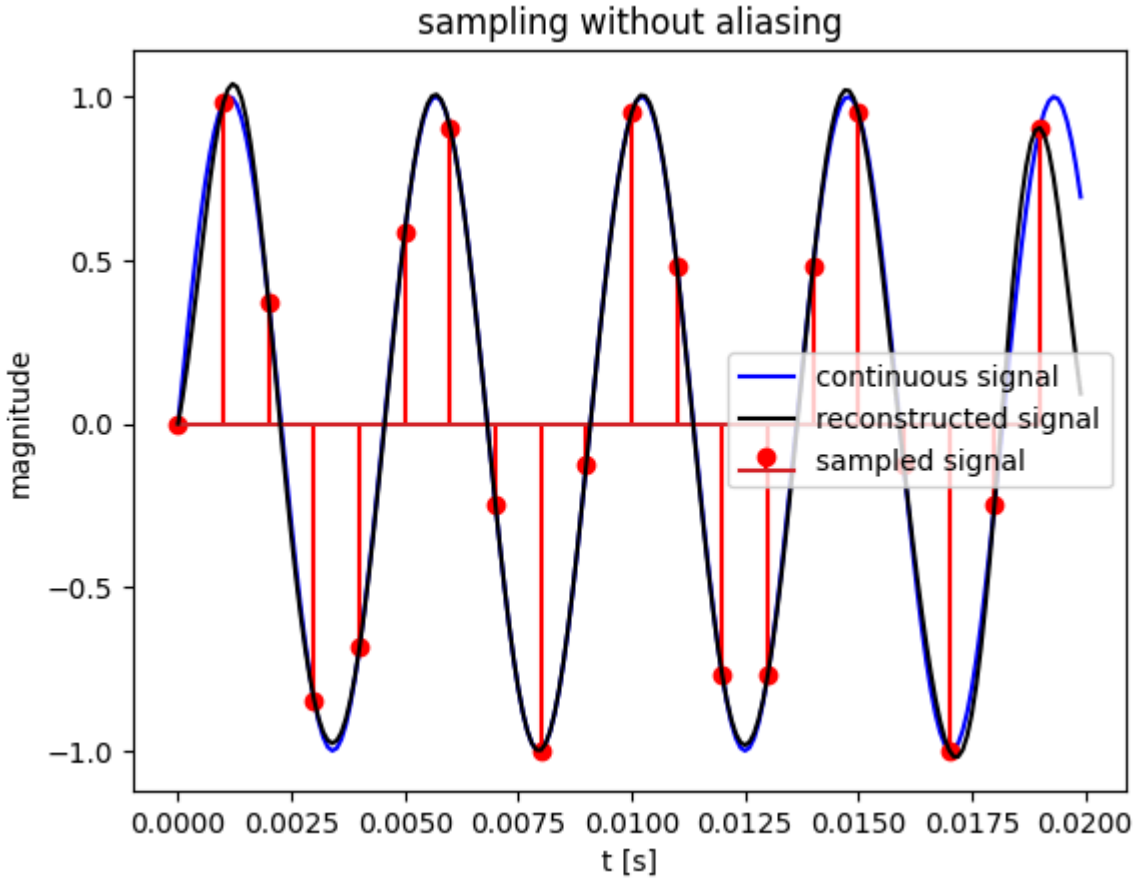
If this sampling theorem is violated, the resulting error is called alias.

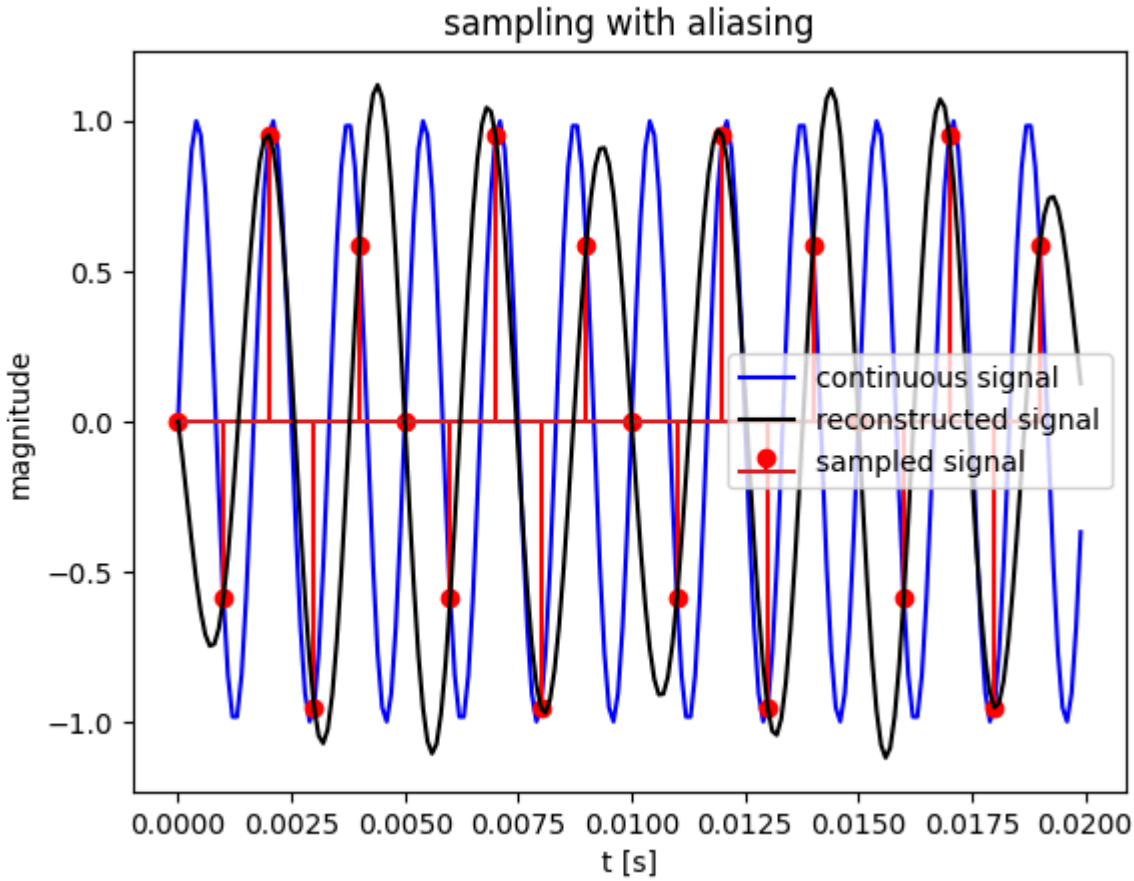
In the following, the continuous signal is sampled. After that, a reconstruction of the continuous signal based on the sampled signal is done. For audio purposes, the reconstruction is done by the perfect lowpass as an interpolation filter.

[3]

```
def TimeContinuousFunction(t, FrequencyInHertz = 220):
    return np.sin(2*np.pi*FrequencyInHertz*t)

def InterpolationFilter(x):
    return np.sinc(x)

def ReconstructedSignal(s_a, t, r, InterpolationFilterFunctionPointer =
InterpolationFilter):
    s_tilde = np.zeros(t.shape)
    for n in range(s_a.shape[0]):
        h = InterpolationFilterFunctionPointer(t*r - n)
        s_tilde += s_a[n] * h
    return s_tilde

r = 1000 # samplerate in Hertz
UpsamplingFactor = 10
t = np.arange(200)/(UpsamplingFactor*r) # simulation of time continuous function by
ten times faster sampling
s = TimeContinuousFunction(t)

t_a = t[0::UpsamplingFactor]
s_a = TimeContinuousFunction(t_a)

plt.plot(t  , s  , 'b', label='continuous signal')
plt.stem(t_a, s_a, 'r', label='sampled signal')
plt.plot(t, ReconstructedSignal(s_a, t, r), 'k', label='reconstructed signal')
plt.xlabel('t [s]')
plt.ylabel('magnitude')
plt.title('sampling without aliasing')
plt.legend()
plt.show()

FrequencyInHertz = 600
t = np.arange(200)/(UpsamplingFactor*r)
s = TimeContinuousFunction(t, FrequencyInHertz)
s_a = TimeContinuousFunction(t_a, FrequencyInHertz)
plt.plot(t  , s  , 'b', label='continuous signal')
plt.stem(t_a, s_a, 'r', label='sampled signal')
plt.plot(t, ReconstructedSignal(s_a, t, r), 'k', label='reconstructed signal')
plt.xlabel('t [s]')
plt.ylabel('magnitude')
```

```
plt.title('sampling with aliasing')
plt.legend()
plt.show()
```





Downsampling

The sampling theorem states, that the following condition must be met, if alias should be avoided:

$f_c \leq \frac{1}{2T}$

In the following, we assume that the highest frequency f_c of a signal $x(n)$ is much less than half of the sampling rate:

$f_c \ll \frac{1}{2T}$

In this case, it is possible to reduce the sampling rate $r = \frac{1}{T}$ by a factor N:

$N \cdot f_c \leq \frac{1}{2T}$

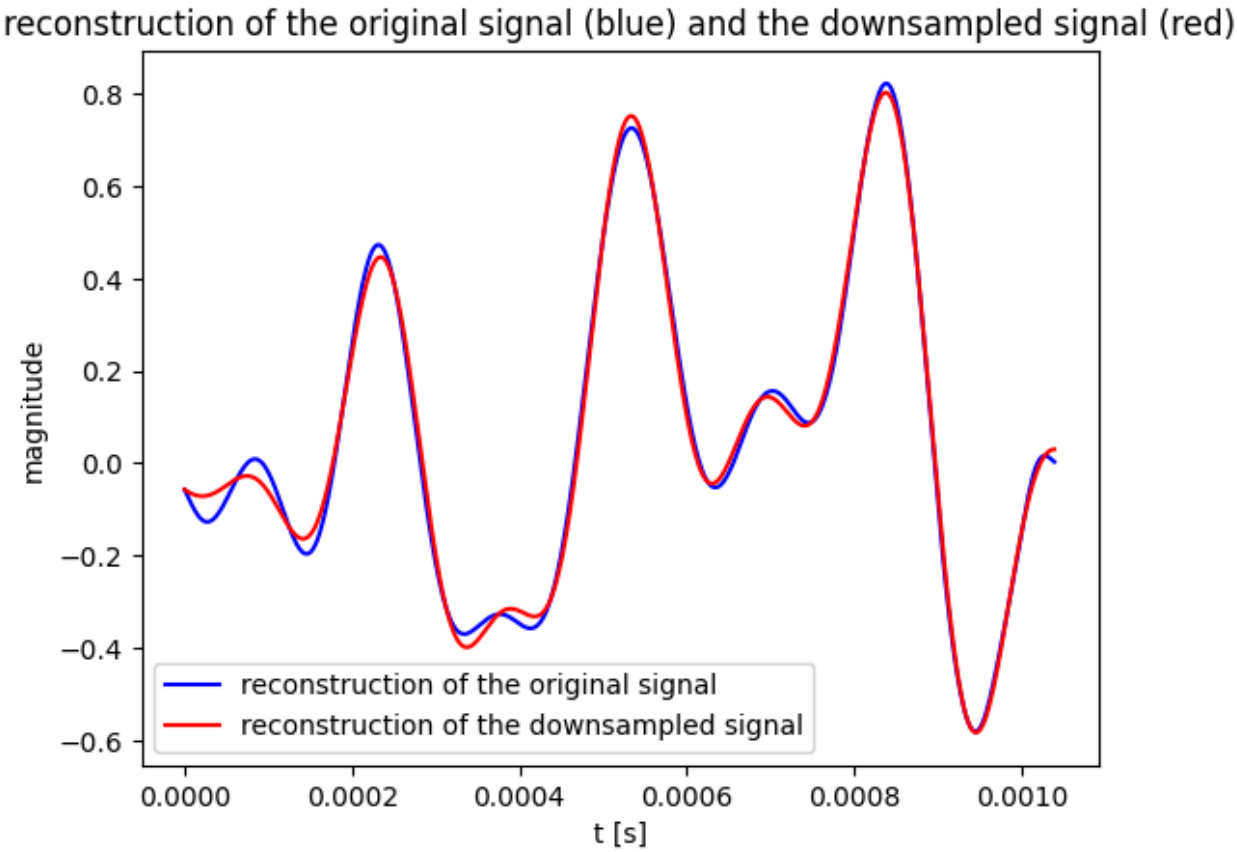
$\rightarrow N \leq \frac{1}{2Tf_c} = \frac{r}{2f_c}$

This reducing of sampling rate is called downsampling. If N is an integer value, the downsampling is a very simple algorithm:

$x_{\text{downsampled}}(n) = x\left(N \cdot n\right)$

[4]

```
r = 48000
x = np.random.randn(50)
fc = 8000
t = np.arange(x.shape[0]) / r
y = np.zeros(x.shape)
for n in range(x.shape[0]):
    y += x[n] * np.sinc(2*fc*(t - n/r)) * 2 * fc / r

N = 10
t = np.arange(y.shape[0] * N) / (r*N)
z1 = ReconstructedSignal(y, t, r)
DownSamplingFactor = r // (2*fc)

y_Downsampled = y[0::DownSamplingFactor]
z2 = ReconstructedSignal(y_Downsampled, t, r / DownSamplingFactor)
plt.plot(t, z1, 'b', label='reconstruction of the original signal')
plt.plot(t, z2, 'r', label='reconstruction of the downsampled signal')
plt.title('reconstruction of the original signal (blue) and the downsampled signal
(red)')
plt.legend()
plt.xlabel('t [s]')
plt.ylabel('magnitude')
plt.show()
SNR = 10*np.log10(np.sum(z1**2) / np.sum((z1-z2)**2))
print('SNR = ', SNR, ' dB')
```




```
SNR =  24.20135762912092  dB
```

Logarithmic level

In the following, two logarithmic levels are introduced, the dB SPL of the analogue world referencing to physical sound pressure and the dB FS of the digital world referencing to the overload point.

dB Sound pressure level

The human ear can analyse a wide range of sound pressure. Therefore, soundlevel is usually described as a logarithmic value, e.g. dB SPL (Sound pressure level):

$$L = 20 \log_{10} \frac{p_{\text{eff}}}{20\mu\text{Pa}} \text{ dB SPL}$$

with p_{eff} corresponding to the RMS of the sound pressure. 0 dB SPL corresponds to a silent sleeping room, where nothing can be heard. 140 dB SPL is a starting jet.

dB Full Scale

dB FS is normalized to the maximum possible amplitude of a digital signal. Assuming the maximum possible magnitude is A (usually, A corresponds to the overload point in digital domain). A sinus with the magnitude A has a RMS of $\frac{A}{\sqrt{2}}$ and a power of $P = \frac{A^2}{2}$. This sinus is defined as 0 dB FS. An arbitrary signal $x(n)$ with a power of

$$P = \frac{1}{N} \sum_{n=0}^{N-1} x^2(n)$$

has a level of

$$L = 10 \log_{10} \frac{P}{\frac{A^2}{2}} \text{ dB FS} = 10 \log_{10} \frac{2 \cdot P}{A^2} \text{ dB FS}$$

The digital output of a AD converter is usually normalized to a maximum possible magnitude of $A = 1$.

Calibration

Calibration can be done by the measurement of dB SPL in physical domain and dB FS in digital domain of the same signal. The calibration can be defined by a parameter a:

$$L_{\text{dB FS}} = a + L_{\text{dB SPL}}$$

E.g. the teachers Smartphone has a calibration factor of $a \approx 115$ dB (measured in January 2023).

Quantisation noise

Digital values are stored with a certain resolution due to quantisation of the samples. This quantisation corresponds on the lowest level to a rounding of continuous values to discrete magnitudes with a resolution of w bits. This rounding can be interpreted as additional noise in the recording if three conditions are fullfilled:

The signal has a power much higher than the power of the quantisation noise.

The signal frequency is high enough, such that neighbouring magnitudes has statistically independent quantisation noise and

the signal is not periodic with a period-length $T_0 = \frac{n}{r}$, with n being an integer.

For condition 1), the sensitivity of the sensor should be high enough, such that the signal is recorded at a reasonable level in dB FS.

Condition 2) is for audio not really a problem, because frequencies below roughly 16 Hz are unhearable for humans. These very low frequencies can therefore be filtered out without any information loss for humans.

Condition 3) is only a problem for artificial test signals (e.g. a 1 KHz signal). In normal recording situations, the input signal has a wide range of frequencies.

If the conditions 1) to 3) are fullfilled, the quantisation noise for w Bits can be interpreted as white noise with a Power of

$$P_E = \frac{\Delta^2}{12}$$

with

$$\Delta = \frac{2A}{2^w}$$

Dithering

If at least one of the three conditions for the white noise assumption of quantisation noise is not fullfilled, dithering should be applied to the digital signal after analogue digital conversion:

Dithering adds random noise at the level of the lowest bit to the signal. By this, the quantisation noise becomes white again.

[5]

```
r = 48000
w = 8
factor = 2**(w-1) # one bit is reserved for the sign
f = 1000
t = np.arange(r) / r
x = np.sin(2*np.pi*f*t) # periodic signal with a period length of 1 ms, violating
the condition 3)
x_quantised =np.round(x*factor) / factor
DitheringNoise = (np.random.rand(x.shape[0]) - 0.5) / factor # dithering noise at
the lowest bit
y = x_quantised + DitheringNoise
y_quantised = np.round(y*factor) / factor
ErrorByDithering = np.sum((x_quantised - y_quantised)**2)
print('Error due to dithering: ', ErrorByDithering)
```

```
Error due to dithering:  0.0
```

Programming exercise:

Fill in the missing code in the following procedures:

EvaluatePowerOfSignal

EvaluateSNROfQuantization

EvaluateSNR

EvaluatedBFullScale

EvaluatedBSoundPressureLevel

[7]

```
import unittest



def EvaluatePowerOfSignal(x):
    x = np.asarray(x)  # sicherstellen, dass x ein NumPy-Array ist
    P_S = np.mean(x**2)  # mittlere Leistung berechnen
    return P_S



def EvaluateSNROfQuantization(w):
    A = 1.0  # Übersteuerungspunkt
    x = np.random.rand(10000) * 2 * A - A  # gleichverteiltes Testsignal im Bereich
[-A, A]
    P_S = EvaluatePowerOfSignal(x)  # Signal-Leistung
    delta = (2*A)/(2**w)  # Quantisierungsintervall
    P_N = (delta**2)/12  # Leistung des Quantisierungsrauschens
    SNR = 10*np.log10(P_S / P_N)  # SNR in dB
    return SNR
```

```
def EvaluateSNR(x, y):
    x = np.asarray(x)  # sicherstellen, dass x ein NumPy-Array ist
    y = np.asarray(y)  # sicherstellen, dass y ein NumPy-Array ist
    error = x - y  # Fehler berechnen
    P_S = EvaluatePowerOfSignal(x)  # Signal-Leistung
    P_N = EvaluatePowerOfSignal(error)  # Fehler-Leistung
    SNR = 10*np.log10(P_S / P_N)  # SNR in dB
    return SNR



def EvaluatedBFullScale(A, x):
    x = np.asarray(x)  # sicherstellen, dass x ein NumPy-Array ist
    P = EvaluatePowerOfSignal(x)  # Signal-Leistung
    L_dBFS = 10 * np.log10(2 * P / (A**2))  # Pegel in dB FS
    return L_dBFS



def EvaluatedBSoundPressureLevel(x_inPascal):
    x_inPascal = np.asarray(x_inPascal)  # sicherstellen, dass x ein NumPy-Array ist
    p_eff = np.sqrt(EvaluatePowerOfSignal(x_inPascal))  # Effektivwert (RMS)
    L_SPL = 20 * np.log10(p_eff / (20e-6))  # Schalldruckpegel in dB SPL
    return L_SPL



class TestProgrammingExercise(unittest.TestCase):

    def test_SNRMonotonicallyIncreasing(self):
        w1 = np.random.randint(20)
        w2 = w1 + 1
        self.assertGreater(EvaluateSNROfQuantization(w2),
EvaluateSNROfQuantization(w1))

    def test_SNRSlope(self):
        for w in range(16):
            self.assertAlmostEqual(EvaluateSNROfQuantization(w), w*6, delta = 0.5)

    def test_EvaluatedBFullScale(self):
        A = np.random.rand(1)
        x = A*np.sin(2*np.pi*440*np.arange(10000)/16000)
        self.assertAlmostEqual(EvaluatedBFullScale(A, x), 0.0, delta = 0.5)
        self.assertAlmostEqual(EvaluatedBFullScale(A, x/np.sqrt(2)), -3.0, delta =
0.5)

    def test_EvaluatedBSoundPressureLevel(self):
        a = np.random.rand(1)
        x_inPascal = a*np.sin(2*np.pi*440*np.arange(10000)/16000)
        self.assertAlmostEqual(EvaluatedBSoundPressureLevel(x_inPascal),
20*np.log10(a) + 90.969, delta = 0.5)

    def test_EvaluatePowerOfSignalSinus(self):
        A = np.random.rand(1)
```

```
        x = A*np.sin(2*np.pi*440*np.arange(10000)/16000)
        self.assertAlmostEqual(EvaluatePowerOfSignal(x), A**2/2, delta = 0.01)

    def test_EvaluatePowerOfSignalUniformNoise(self):
        A = np.random.rand(1)
        x = 2*A*np.random.rand(10000)-A
        self.assertAlmostEqual(EvaluatePowerOfSignal(x), (2*A)**2/12, delta = 0.01)

    def test_EvaluatePowerOfSignalGaussianNoise(self):
        s = np.random.rand(1)
        x = s*np.random.randn(10000)
        self.assertAlmostEqual(EvaluatePowerOfSignal(x), s**2, delta = 0.1)

    def test_EvaluateSNRErrorEqualSignal(self):
        x = np.random.randn(10000)
        e = np.random.randn(10000)
        y = x + e
        self.assertAlmostEqual(EvaluateSNR(x, y), 0.0, delta = 0.5)

    def test_EvaluateSNRSmallError(self):
        factor = 1e-3
        x = np.random.randn(10000)
        e = np.random.randn(10000) * factor
        y = x + e
        self.assertAlmostEqual(EvaluateSNR(x, y), -20*np.log10(factor), delta = 3.0)

# https://stackoverflow.com/questions/40172281/unit-tests-for-functions-in-a-
jupyter-notebook
unittest.main(argv=['-f'], verbosity=2, exit=False)
```

Collapse

```
test_EvaluatePowerOfSignalGaussianNoise
(__main__.TestProgrammingExercise.test_EvaluatePowerOfSignalGaussianNoise) ... ok
test_EvaluatePowerOfSignalSinus
(__main__.TestProgrammingExercise.test_EvaluatePowerOfSignalSinus) ... ok
test_EvaluatePowerOfSignalUniformNoise
(__main__.TestProgrammingExercise.test_EvaluatePowerOfSignalUniformNoise) ... ok
test_EvaluateSNRErrorEqualSignal
(__main__.TestProgrammingExercise.test_EvaluateSNRErrorEqualSignal) ... ok
test_EvaluateSNRSmallError
(__main__.TestProgrammingExercise.test_EvaluateSNRSmallError) ... ok
test_EvaluatedBFullScale (__main__.TestProgrammingExercise.test_EvaluatedBFullScale)
... ok
test_EvaluatedBSoundPressureLevel
(__main__.TestProgrammingExercise.test_EvaluatedBSoundPressureLevel) ... ok
test_SNRMonotonically Increasing
(__main__.TestProgrammingExercise.test_SNRMonotonicallyIncreasing) ... ok
test_SNRSlope (__main__.TestProgrammingExercise.test_SNRSlope) ... ok

----------------------------------------------------------------------
Ran 9 tests in 0.017s

OK
```

```
Collapse
test_EvaluatePowerOfSignalGaussianNoise
(__main__.TestProgrammingExercise.test_EvaluatePowerOfSignalGaussianNoise) ... ok
test_EvaluatePowerOfSignalSinus
(__main__.TestProgrammingExercise.test_EvaluatePowerOfSignalSinus) ... ok
test_EvaluatePowerOfSignalUniformNoise
(__main__.TestProgrammingExercise.test_EvaluatePowerOfSignalUniformNoise) ... ok
test_EvaluateSNRErrorEqualSignal
(__main__.TestProgrammingExercise.test_EvaluateSNRErrorEqualSignal) ... ok
test_EvaluateSNRSmallError
(__main__.TestProgrammingExercise.test_EvaluateSNRSmallError) ... ok
test_EvaluatedBFullScale (__main__.TestProgrammingExercise.test_EvaluatedBFullScale)
... ok
test_EvaluatedBSoundPressureLevel
(__main__.TestProgrammingExercise.test_EvaluatedBSoundPressureLevel) ... ok
test_SNRMonotonically Increasing
(__main__.TestProgrammingExercise.test_SNRMonotonically Increasing) ... ok
test_SNRSlope (__main__.TestProgrammingExercise.test_SNRSlope) ... ok

----------------------------------------------------------------------
Ran 9 tests in 0.017s

OK
```

```
<unittest.main.TestProgram at 0x2c6fd9dca50>
```

Exam preparation

A quantizer with $w = 16$ bit is used. Give the amount of data for 70 minutes of recording on an audio cd. Hint: Audio cd are recorded in stereo mode.

Evaluate the level in dB SPL of a sinus with a magnitude of 0.1 Pa. This sinus is digitalized to a digital signal $x(n) = 0.25 \cdot \sin\left(2\pi f \frac{n}{r}\right)$. The maximum possible magnitude of this AD converter is $A = 1.0$. Evaluate the corresponding level dB FS and the calibration factor a.

Evaluate the SNR for a signal with -12 dB FS with quantization noise for $w = 16$ bit. $A = 1.0$ is still valid.

A signal $x(t) = a \cdot \cos(2\pi f t)$ is given, with $f = 2$ kHz. $x(t)$ has a level of 75 dB SPL. Evaluate the parameter a. Is it possible to evaluate the sign of a by the given informations? Evaluate the level in dB SPL if the frequency is doubled.

What kind of function $y = f(x)$ results in a linear plot for a logarithmic x-axis, for a logarithmic y-axis and for a double-logarithmic plot?

A signal has a calibration factor of $a = -120$ dB. Evaluate the Overload Point in dB SPL. Assume the signal to be a sinus.

Evaluate the level in dB FS of the quantisation noise for $w = 16$ Bits.

Give the mathematical description of a digital signal, which corresponds to the highest possible level in dB FS. What is the corresponding level in dB FS?

A time continuous signal $x(t) = \cos(7 \cdot t)^2$ is sampled by $r = 10$ Hz. Evaluate the first five samples.

A time discrete signal sampled with $r = 32$ kHz is filtered by a lowpass with a cutoff frequency of $f_c = 3400$ Hz. What is the lowest necessary sampling frequency after the lowpass. What is the maximum allowed integer downsampling factor? Corresponds the signal after the lowpass to wideband speech or to narrowband speech?

A microfone signal $y(n)$ is recorded with $r = 48$ kHz. Afterwards, $y(n)$ is filtered by a bandpass $h(n)$ with cutoff frequencies $f_1 = 100$ Hz and $f_2 = 7000$ Hz: $z(n) = h(n) * y(n)$. Does $z(n)$ has a DC component? Are human voices understandable in $z(n)$? What are the highest frequencies in $y(n)$ and $z(n)$?

A signal $x(t) = \cos(500 \cdot t)$ is sampled with $r = 200$ Hz. Does this sampling cause alias?

Lösungen zu den Übungsfragen

Datenmenge für 70 Minuten Audio-CD (Stereo, 16 Bit):
$70\,\text{min} \times 60\,\text{s/min} \times 44100\,\text{Hz} \times 16\,\text{Bit} \times 2\,\text{Kanäle} = 5\,929\,920\,000\,\text{Bit} = 741.24\,\text{MB}$

dB SPL eines Sinus mit 0.1 Pa:
$L_{\text{SPL}} = 20\log_{10}(0.1/20\mu\text{Pa}) \approx 94\,\text{dB SPL}$
dB FS für $x(n) = 0.25\sin(\ldots)$, $A = 1.0$:
$L_{\text{dBFS}} = 10\log_{10}(2 \cdot (0.25^2/2)/1^2) = -12\,\text{dB FS}$
Kalibrierfaktor:
$a = L_{\text{dBFS}} - L_{\text{SPL}} = -12 - 94 = -106\,\text{dB}$

SNR für -12 dB FS, $w = 16$:
$SNR_Q = 6 \cdot w = 96\,\text{dB}$
$SNR_{\text{gesamt}} = -12\,\text{dB FS} - L_{\text{Q,FS}}$
$L_{\text{Q,FS}} = -96\,\text{dB FS}$
$\Rightarrow SNR = 84\,\text{dB}$

a für 75 dB SPL:
$a = 20\mu\text{Pa} \cdot 10^{75/20} \approx 112.2\,\text{Pa}$
Vorzeichen: Nicht bestimmbar (nur Betrag).
Frequenz verdoppelt: Pegel bleibt gleich (Amplitude bestimmt Pegel, nicht Frequenz).

Funktionen für lineare Plots:

- log-x: $y = a\log(x) + b$
- log-y: $y = b \cdot x^a$
- doppelt-log: $y = c \cdot x^a$ (Potenzfunktion)

Übersteuerungspunkt bei $a = -120$ dB:
$L_{\text{dBFS}} = a + L_{\text{SPL}}$, für $L_{\text{dBFS}} = 0$
$\Rightarrow L_{\text{SPL}} = 120\,\text{dB SPL}$

Quantisierungsrauschen $w = 16$:
$L_{\text{Q,FS}} = -6.02 \cdot w = -96.3\,\text{dB FS}$

Maximaler dB FS:
$x(n) = A \cdot \sin(\ldots), L_{\text{dBFS}} = 0\,\text{dB FS}$

Erste 5 Samples $x(t) = \cos^2(7t), r = 10$ Hz:
$t_n = n/10, x(n) = \cos^2(7 \cdot n/10)$ für $n = 0..4$

Nach Tiefpass $f_c = 3400$ Hz:
$2f_c = 6800$ Hz, also $r_{\text{min}} = 6800$ Hz
$N_{\text{max}} = \lfloor 32000/6800 \rfloor = 4$
Wideband oder Narrowband: Wideband (Grenze: 3400 Hz)

Bandpass 100 Hz bis 7000 Hz:

- $z(n)$ hat keine DC-Komponente
- Sprache verständlich (Stimmfrequenzen enthalten)
- $y(n)$: bis 24 kHz, $z(n)$: bis 7 kHz

Alias bei $x(t) = \cos(500t), r = 200$ Hz:
$500/(2\pi) \approx 79.6$ Hz, $r/2 = 100$ Hz, also kein Alias, da $f < r/2$.

Summary

After working with this Jupyter Notebook you should be able to explain the following topics:

- In which cases is the overload point A relevant for your measurement?
- In which case is the number of bits w relevant for your measurement?
- In which case is the sampling rate r relevant for your measurement?
- What is the sampling theorem?
- What is aliasing?
- What are the difference between dB SPL and dB FS?
- When do you want to implement dithering?
- How does the quantisation noise affects your measurement?

Antworten zu den wichtigsten Themen

Übersteuerungspunkt A: Relevant, wenn du wissen willst, ab welcher Amplitude dein System verzerrt (Clipping). Alle Messwerte müssen im Bereich $-A$ bis A liegen, sonst gehen Informationen verloren.

Anzahl der Bits w: Bestimmt die Auflösung der Quantisierung. Je mehr Bits, desto feiner die Abstufung und desto geringer das Quantisierungsrauschen.

Abtastrate r: Gibt an, wie viele Werte pro Sekunde aufgenommen werden. Sie muss mindestens doppelt so hoch wie die höchste Signal-Frequenz sein (siehe Abtasttheorem), um Informationsverlust (Aliasing) zu vermeiden.

Abtasttheorem: Ein Signal kann nur dann fehlerfrei rekonstruiert werden, wenn die Abtastrate mindestens doppelt so hoch wie die höchste vorkommende Frequenz ist: $f_c \leq r/2$.

Aliasing: Tritt auf, wenn die Abtastrate zu niedrig ist. Höhere Frequenzen erscheinen dann als niedrigere Frequenzen im abgetasteten Signal und verfälschen das Ergebnis.

Unterschied dB SPL und dB FS:

dB SPL (Sound Pressure Level): Bezieht sich auf den physikalischen Schalldruck (Vergleich zu 20 µPa, der Hörschwelle).

dB FS (Full Scale): Bezieht sich auf den maximal möglichen Wert im digitalen System (0 dB FS = maximaler Wert).

Wann Dithering? Wenn die Bedingungen für weißes Quantisierungsrauschen nicht erfüllt sind (z.B. bei sehr leisen oder periodischen Signalen), wird Dithering eingesetzt, um das Rauschen zu „entkoppeln“ und hörbare Artefakte zu vermeiden.

Einfluss des Quantisierungsrauschens: Quantisierungsrauschen ist ein unvermeidlicher Fehler durch die Digitalisierung. Es kann als weißes Rauschen modelliert werden, solange das Signal ausreichend groß und nicht zu regelmäßig ist. Je mehr Bits, desto geringer das Rauschen und desto besser die Signalqualität.